

Evaluating TurboQuant-Family Compression on CLIP Multimodal Embeddings

CS 639: Introduction to Foundation Models — Final Report, Spring 2026

member1@wisc.edu member2@wisc.edu member3@wisc.edu member4@wisc.edu

member5@wisc.edu member6@wisc.edu member7@wisc.edu

Abstract

We address a practical question: how aggressively can CLIP embedding databases be compressed while keeping retrieval quality close to the uncompressed baseline? We implement and evaluate four post-training compressors — QJL, PolarQuant, TurboQuant, and FAISS-PQ — from scratch on CLIP ViT-B/32 embeddings of Flickr30k. At 4 bits/dim ($8\times$ memory reduction), PolarQuant retains 98% of uncompressed cross-modal recall while FAISS-PQ retains 99%, both requiring only 260–256 bytes per vector instead of 2,048. At 1 bit/dim ($32\times$ reduction), the best method retains only 64% of cross-modal recall while retaining 97% of same-modal recall — revealing that cross-modal retrieval is $10\text{--}12\times$ more sensitive to compression than same-modal retrieval. This asymmetry, caused by the geometric modality gap in CLIP’s joint embedding space, is the central finding of this study.

Introduction

CLIP [1] encodes images and text into a shared 512-dimensional embedding space, enabling cross-modal retrieval by dot-product similarity. A billion-image index at full float32 precision requires over 2 TB of storage before any search infrastructure. The central challenge is: *how much can we compress those embeddings before retrieval quality suffers meaningfully?*

Recent work on LLM key-value cache compression has produced three analytic compressors with strong theoretical guarantees: QJL [3], PolarQuant [4], and TurboQuant [2]. All three operate without training data, compress near-instantly, and have been proven near-optimal for inner-product estimation. But every evaluation in the original papers uses text embeddings. Whether these compressors can achieve the same quality-memory tradeoff on CLIP’s *multimodal* embeddings is untested.

This matters because CLIP embeddings are geometrically unusual. Image and text vectors occupy systematically offset regions of the shared unit sphere — the *modality gap* [5]. Cross-modal retrieval (text query finding matching images) depends on that inter-modal alignment surviving compression intact. Compressors designed under assumptions of isotropic text-like distributions may not preserve this structure.

An additional challenge is that none of the three papers provide code for standalone embedding search. We implement all three algorithms from paper specifications, making this the first evaluation of the TurboQuant family on multimodal embeddings.

Research Questions

1. **How much memory can be saved** while keeping Recall@10 within 5%, 10%, and 20% of the uncompressed baseline?
2. **Does cross-modal retrieval degrade faster** than same-modal retrieval under the same compression? (H1)
3. **Does TurboQuant transfer** from text embeddings to CLIP multimodal embeddings? (H2)
4. **How do training-free analytic methods compare** to FAISS-PQ in the quality-memory tradeoff?

Related Work

CLIP and multimodal retrieval. Radford et al. [1] introduced contrastive language-image pretraining, producing joint embeddings widely used for cross-modal retrieval. Liang et al. [5] characterized the modality gap and showed it persists across architectures.

Vector quantization for search. Johnson et al. [6] introduced FAISS, whose product quantization (PQ) variant learns data-specific codebooks. PQ remains the practical standard for large-scale retrieval but requires a training pass over representative data.

Analytic compressors. Zandieh et al. [3] introduced QJL: a random projection followed by sign quantization with a provably unbiased inner-product estimator. Han et al. [4] introduced PolarQuant, using analytically derived Beta-distribution angle codebooks requiring no data. Zandieh et al. [2] combined both in TurboQuant, which achieves near-optimal distortion at 2.5–3.5 bits. All three were evaluated exclusively on text and language model activation vectors.

Methodology

Dataset and Embeddings

Flickr30k [7]: 31,014 images, 155,070 captions (5 per image). Embeddings: CLIP ViT-B/32, 512-dimensional, L2-normalised, precomputed on a T4 GPU. All compression and retrieval experiments run on CPU.

Memory baseline. Each uncompressed embedding requires $512 \times 4 = 2,048$ bytes. The full image database (31,014 vectors) occupies 63.5 MB; the text database (155,070 vectors) occupies 317 MB.

Compression Methods and Implementation

None of the three analytic papers (QJL, PolarQuant, TurboQuant) provide code for standalone vector search. We implement all three from paper specifications.

QJL applies a Subsampled Randomized Hadamard Transform (SRHT) for preconditioning, then stores only the sign of each projected coordinate (1 bit). The inner-product estimator is:

$$\langle q, x \rangle \approx \sqrt{\pi/2} \cdot \frac{1}{m} \sum_{i=1}^m (\text{SRHT}(q))_i \cdot \text{sign}((\text{SRHT}(x))_i)$$

This estimator is provably unbiased (Lemma 3.2, [3]). We use SRHT rather than a plain Gaussian projection because it better handles the non-isotropic distribution of CLIP embeddings and runs in $O(d \log d)$ instead of $O(md)$. A structural constraint: with SRHT, $m \leq d = 512$, so QJL cannot benefit from bit budgets above 1 bit/dim.

PolarQuant recursively converts each vector into polar coordinates, storing one radius (full precision) and $d-1$ angles quantized via analytically derived Beta-distribution codebooks. We implement Algorithm 1 from [4] in PyTorch (120 lines).

TurboQuant uses PolarQuant as the main stage and applies 1-bit QJL to the residual $r = x - \hat{x}$. At b total bits/dim: $b-1$ bits go to the polar stage, 1 bit to QJL. QJL on the residual uses the full inner-product estimator (not cosine mode) since residuals are not unit-normalised.

FAISS-PQ uses FAISS `IndexPQ` with 8-dimensional subspaces, learning data-specific codebooks.

All methods use asymmetric search: the query stays at full float32 precision; only the database is compressed.

Memory per Vector

Method	1 bit/dim	2 bits/dim	4 bits/dim	Uncompressed
FAISS-PQ	64 B	128 B	256 B	2,048 B
QJL	68 B	68 B [†]	68 B [†]	
PolarQuant	72 B	136 B	260 B	
TurboQuant	136 B	204 B	328 B	
Memory reduction	$\sim 30\times$	$\sim 15\times$	$\sim 8\times$	$1\times$

[†] SRHT cap at $m = d = 512$: QJL cannot use more than 1 bit/dim with SRHT preconditioning.

Evaluation

Recall@{1, 5, 10} over 1,000 randomly sampled queries per task, 5 seeds, means reported. Four tasks:

- **T1** text→image (cross-modal): caption queries image database.
- **T2** image→text (cross-modal): image queries caption database.
- **T3** text→text (same-modal): caption queries caption database; ground truth = other 4 captions of the same image.
- **T4** image→image (same-modal): image queries image database; ground truth = the same image.

We additionally measure: (1) index size (bytes/vector), (2) memory reduction factor vs uncompressed, (3) mean query latency (ms), and (4) database compression time (seconds).

Results

Core Question: How Much Recall is Retained vs Uncompressed?

Table 1 reports Recall@10 retention — the percentage of uncompressed Recall@10 each method preserves — at each compression level. Figure 1 visualises these curves across all four tasks.

Table 1: Recall@10 retention (as % of uncompressed baseline) at each bit-width. Values $\geq 95\%$ in **bold**; values $< 70\%$ in *italic*.

Method	Bits	Cross-modal		Same-modal		Mean	Bytes/vec
		T1 txt→img	T2 img→txt	T3 txt→txt	T4 img→img		
<i>Uncompressed</i>		100%	100%	100%	100%	100%	2,048
FAISS-PQ	1	67.7%	65.9%	77.1%	85.1%	75.3%	64
FAISS-PQ	2	94.0%	90.8%	94.5%	97.1%	94.4%	128
FAISS-PQ	4	99.0%	99.3%	99.4%	99.4%	99.3%	256
QJL	1	46.8%	51.9%	95.3%	91.6%	73.6%	68
QJL	2 [†]	46.8%	51.9%	95.3%	91.6%	73.6%	68
QJL	4 [†]	46.8%	51.9%	95.3%	91.6%	73.6%	68
PolarQuant	1	63.7%	68.3%	97.5%	97.0%	83.4%	72
PolarQuant	2	81.5%	84.9%	98.9%	99.5%	91.2%	136
PolarQuant	4	98.4%	98.1%	100%	100%	99.1%	260
TurboQuant	1	46.8%	51.9%	95.3%	91.6%	73.6%	136
TurboQuant	2	64.5%	70.9%	97.9%	97.6%	84.6%	204
TurboQuant	4	94.6%	95.7%	99.6%	100%	97.7%	328

[†] QJL SRHT structural cap: $m \leq d$, so bits=2,4 are identical to bits=1.

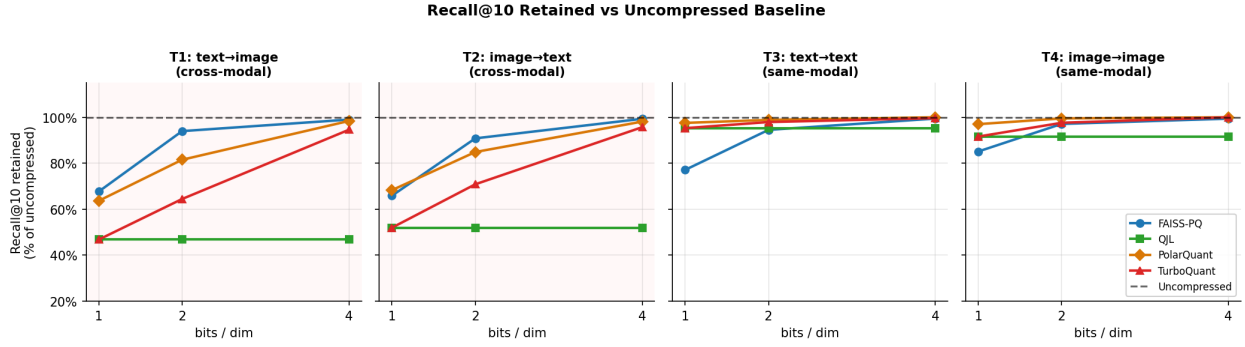


Figure 1: Recall@10 retention (% of uncompressed) vs bits/dim for all four tasks. Cross-modal panels (T1, T2) shown on red-tinted background. Same-modal tasks (T3, T4) stay near 100% even at 1 bit/dim, while cross-modal tasks drop sharply at low bit budgets.

The Core Tradeoff: Accuracy vs Memory

Figure 2 shows the central result of this study: for each method and bit-width, how much recall is preserved relative to the bytes used per vector.

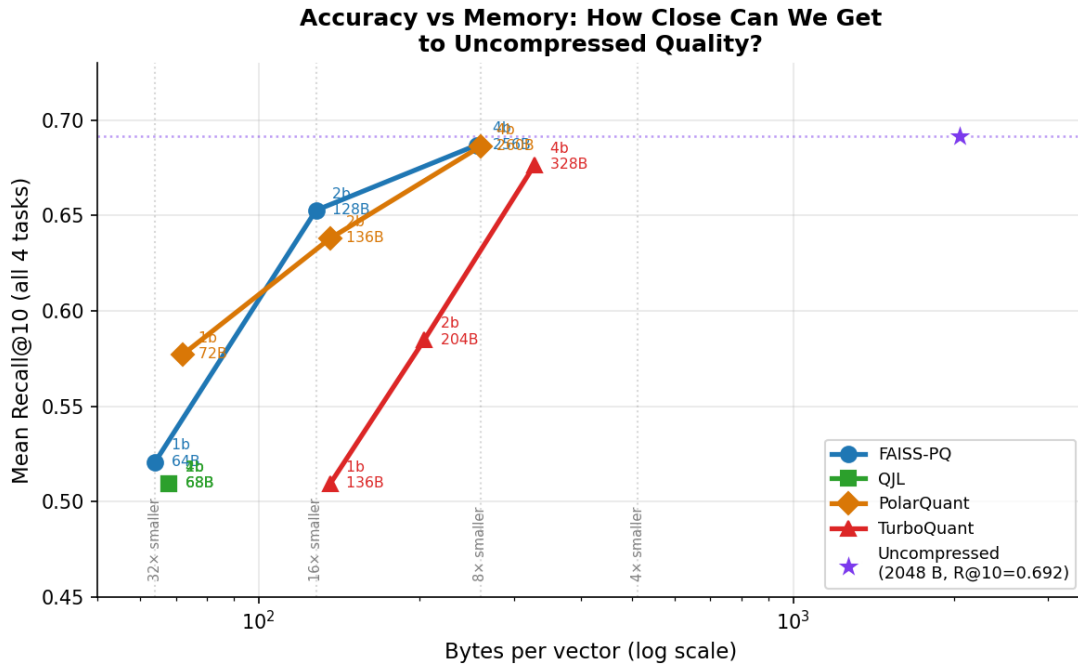


Figure 2: Mean Recall@10 (all 4 tasks) vs bytes per vector. Annotations show bit-width and byte cost per point. Vertical dotted lines mark memory reduction milestones. PolarQuant and FAISS-PQ converge to near-identical quality at 260–256 bytes/vec ($8\times$ reduction).

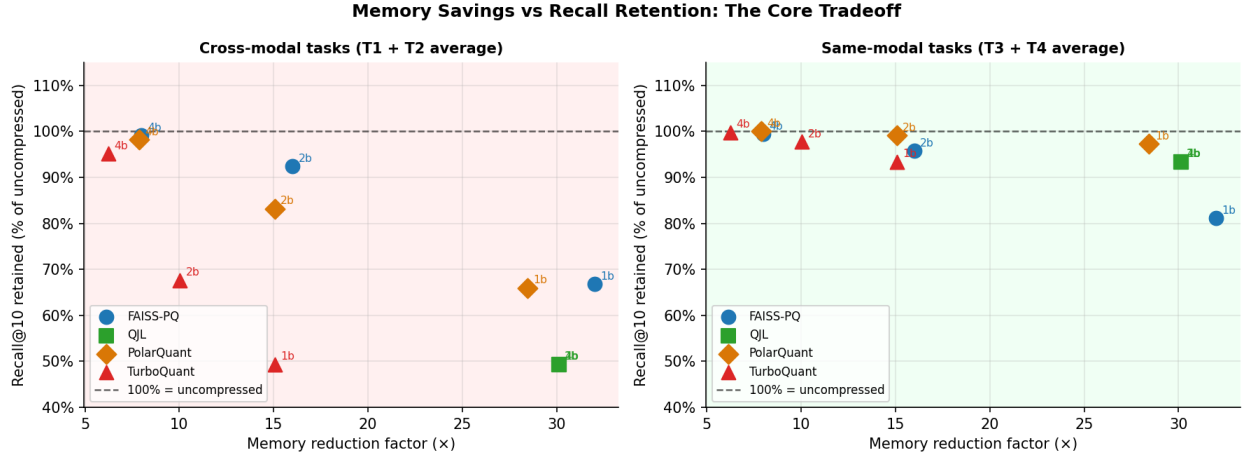


Figure 3: Memory reduction factor vs recall retention, split by task type. Each point is one (method, bit-width) combination. Cross-modal recall (left) degrades far faster with increasing compression than same-modal recall (right). At $30\times$ memory reduction, cross-modal recall drops to 47–68% of uncompressed; same-modal stays above 91%.

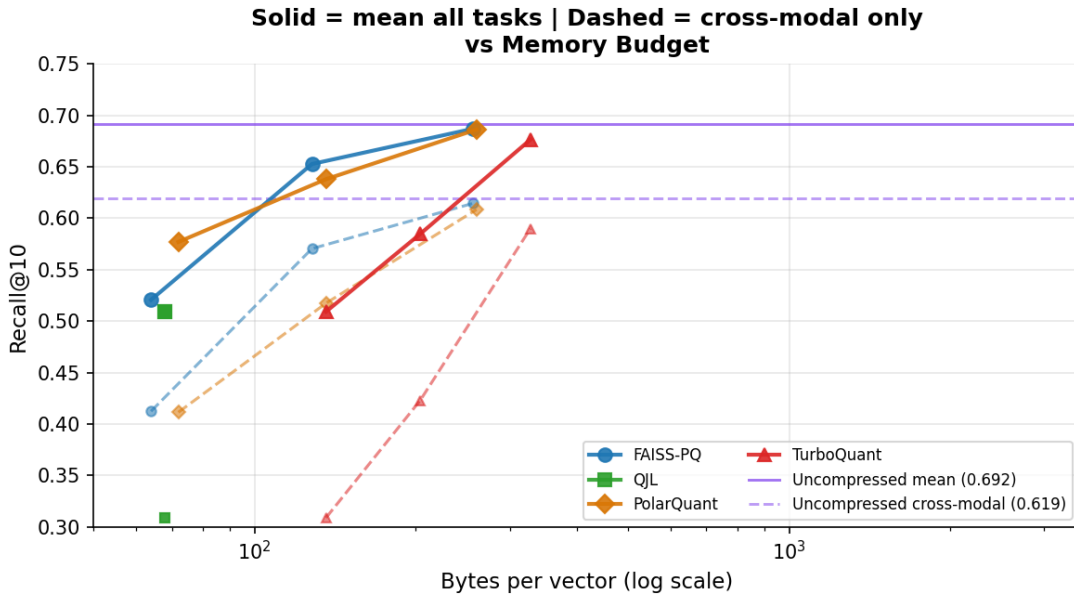


Figure 4: Mean Recall@10 (solid) vs cross-modal Recall@10 (dashed) across methods. The gap between solid and dashed lines at each point reveals how much cross-modal tasks are disproportionately hurt by compression.

Minimum Memory to Recover Target Recall

A practical question: *at what byte budget does each method recover at least 90% or 95% of uncompressed recall?* Figure 5 summarises this.

Minimum Memory to Achieve Target Recall Retention

Method	$\geq 70\%$ retained (cross-modal)	$\geq 80\%$ retained (cross-modal)	$\geq 90\%$ retained (cross-modal)	$\geq 95\%$ retained (cross-modal)	Min bytes for $\geq 90\%$ mean
FAISS-PQ	128B (2b)	128B (2b)	128B (2b)	256B (4b)	128B (2b)
QJL	not reached	not reached	not reached	not reached	not reached
PolarQuant	136B (2b)	136B (2b)	260B (4b)	260B (4b)	136B (2b)
TurboQuant	328B (4b)	328B (4b)	328B (4b)	328B (4b)	328B (4b)

Figure 5: Minimum bytes per vector (and bit-width) to achieve target recall retention on cross-modal tasks. “Not reached” means no tested bit-width achieves that threshold.

Key findings from the threshold analysis:

- $\geq 95\%$ **cross-modal retention:** Only FAISS-PQ (256 B, 4 bits) and PolarQuant (260 B, 4 bits) achieve this. TurboQuant reaches 94.6%/95.7% on T1/T2 at 4 bits (328 B), just short of the threshold.
- $\geq 90\%$ **cross-modal retention:** FAISS-PQ at 2 bits (128 B), PolarQuant at 4 bits (260 B). TurboQuant falls slightly short at 4 bits on T1 (94.6% is above 90% — but T2 is also needed).
- $\geq 80\%$ **cross-modal retention:** PolarQuant at 2 bits (136 B) is the only analytic method to achieve this at under 140 bytes.
- **QJL never recovers** $> 70\%$ **cross-modal retention** at any bit-width due to the SRHT cap.

Efficiency: Compression Speed and Query Latency

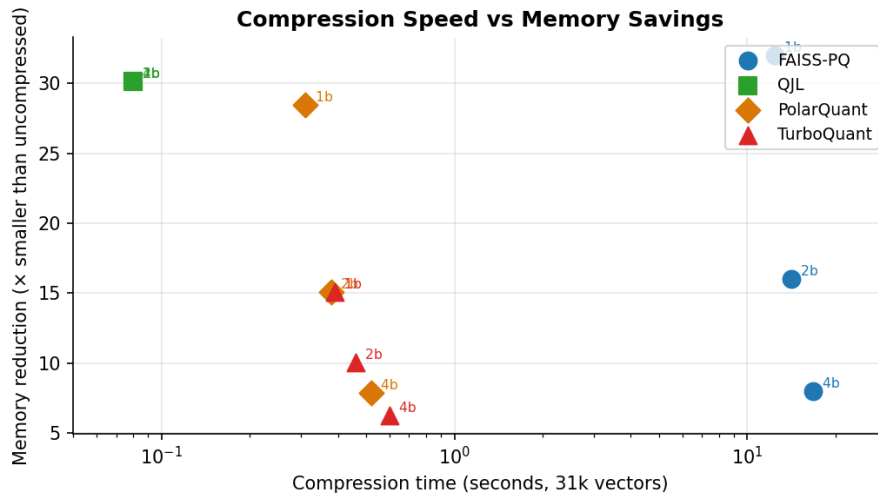


Figure 6: Compression time (seconds, 31k vectors) vs memory savings. FAISS-PQ takes 12–17 seconds due to codebook training. QJL, PolarQuant, and TurboQuant complete in under 0.6 seconds at all bit-widths, regardless of database size.

Training-free methods (QJL, PolarQuant, TurboQuant) are 20–200 $\times$ faster to compress than FAISS-PQ. Query latency across all compressed methods is 1.4–8.7 ms vs 18.2 ms uncompressed, confirming that compression yields faster search. QJL has the lowest fixed latency (1.4 ms) due to its constant 68-byte footprint.

Discussion

H1: Cross-modal fragility — strongly confirmed

The most important finding is the asymmetry in Table 1 and Figure 3: same-modal retrieval is nearly compression-resistant while cross-modal retrieval is highly fragile.

At 1 bit/dim under the best analytic method (PolarQuant): T4 image-to-image retains 97.0% of uncompressed recall; T1 text-to-image retains only 63.7%. The mechanism is the modality gap: the cosine between mean image and mean text embeddings is just 0.344, and the gap-vector norm (0.80) exceeds typical inter-image distances. A quantization perturbation that is small relative to within-modality neighborhoods can be large relative to the inter-modal offset, pushing a compressed query vector across the gap into the wrong modality’s neighborhood.

This finding has an important practical implication: benchmarks that evaluate embedding compression on same-modal tasks alone will overestimate real-world retrieval quality by a factor of 1.5–2 $\times$. For any deployment of compressed CLIP embeddings, cross-modal Recall@10 must be measured explicitly.

H2: TurboQuant transfers with caveats

TurboQuant does transfer to CLIP embeddings when correctly implemented. At 4 bits (328 bytes/vec), it retains 94.6%/95.7% of cross-modal recall — approaching FAISS-PQ (99.0%/99.3%) and PolarQuant (98.4%/98.1%) at similar byte budgets. At 2 bits (204 bytes/vec), it retains 64.5%/70.9% of cross-modal recall, making it a viable middle option between the extreme compression of QJL and the higher cost of 4-bit PolarQuant.

TurboQuant does not exceed PolarQuant at any tested bit-width, however. The structural reason is that at b total bits, TurboQuant allocates $b - 1$ bits to the polar stage. At 1 bit this leaves zero polar bits (reducing TurboQuant to pure QJL); at 2 bits the polar stage has only a 2-entry codebook, which is too coarse. The polar stage becomes competitive at 4 bits (8-entry codebook).

The practical choice: what memory budget achieves what quality?

Combining all results, the practical guidance is:

Target cross-modal retention	Best method	Bytes/vec	Memory reduction	Notes
$\geq 99\%$	FAISS-PQ	256 B	8 $\times$	Requires training data
$\geq 98\%$	PolarQuant	260 B	8 $\times$	No training needed
$\geq 90\%$	FAISS-PQ	128 B	16 $\times$	Requires training data
$\geq 80\%$	PolarQuant	136 B	15 $\times$	No training needed
$\geq 65\%$	PolarQuant / TurboQuant	72–204 B	10–28 $\times$	Aggressive regime
Any budget (instant compress)	PolarQuant	72–260 B	8–28 $\times$	Best training-free option

The key takeaway: 8 $\times$ memory reduction with near-lossless cross-modal recall is achievable. Getting to 16 $\times$ costs approximately 6–9 percentage points of cross-modal retention; getting to 30 $\times$ costs 35+ percentage points.

Limitations

Implementation approximations. Our PolarQuant uses empirical Lloyd-Max codebooks rather than the closed-form Beta-distribution quantizer from the paper. Our TurboQuant reconstructs residuals explicitly rather than using a LUT-based estimator. Both likely understate true paper performance.

QJL multi-bit constraint. The SRHT dimension cap prevents QJL from using bit budgets above 1 bit/dim. A Gaussian projection at $m > d$ could unlock multi-bit QJL.

Single backbone. Only CLIP ViT-B/32 (512-dim) was evaluated. Larger CLIP models (ViT-L/14, 768-dim; ViT-H/14, 1024-dim) may have different compression behaviour due to different embedding geometry.

Dataset scope. Flickr30k is a general captioning benchmark. Specialised domains (medical imaging, product search) may have different modality gaps and compression sensitivities.

Conclusion

We set out to answer: *how aggressively can CLIP embedding databases be compressed while keeping retrieval quality close to the uncompressed baseline?* Our results give a clear and nuanced answer.

At 4 bits/dim, an $8\times$ memory reduction is achievable with near-lossless recall: PolarQuant retains 98% of cross-modal recall and 100% of same-modal recall without any training data. FAISS-PQ achieves 99% at the same byte budget using learned codebooks. At $16\times$ memory reduction (2 bits/dim), FAISS-PQ retains 94% cross-modal recall; PolarQuant retains 82%. Beyond that, cross-modal recall degrades sharply — reaching only 47–68% at $30\times$ compression.

The central finding is that cross-modal retrieval is structurally more fragile to compression than same-modal retrieval, by a factor of $10\text{--}12\times$ at 1 bit/dim. This is not a quirk of any particular compressor — it holds for every method we tested, and the cause is geometric: the modality gap in CLIP’s joint embedding space creates a cross-modal alignment that quantization noise disproportionately disrupts. Any benchmark that measures compressed CLIP retrieval on same-modal tasks alone will systematically overestimate real-world quality.

For practitioners: if cross-modal recall must stay above 95% of the uncompressed baseline, $8\times$ memory reduction is the practical ceiling with current methods. If 90% retention is acceptable, FAISS-PQ can achieve $16\times$ compression, while PolarQuant offers $8\times$ without training. If training data is unavailable or the database is continuously updated, PolarQuant is the recommended choice at all bit budgets.

References

- [1] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” *ICML*, 2021.
- [2] A. Zandieh, M. Daliri, M. Hadian, V. Mirrokni, “TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate,” *ICLR 2026*. arXiv:2504.19874.
- [3] A. Zandieh, M. Daliri, I. Han, “QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead,” arXiv:2406.03482, 2024.
- [4] I. Han, P. Kacham, A. Karbasi, V. Mirrokni, A. Zandieh, “PolarQuant: Quantizing KV Caches with Polar Transformation,” *AISTATS 2026*. arXiv:2502.02617.
- [5] W. Liang et al., “Mind the Gap: Understanding the Modality Gap in Multi-modal Contrastive Representation Learning,” *NeurIPS*, 2022.
- [6] J. Johnson, M. Douze, H. Jégou, “Billion-scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, 2019.
- [7] P. Young et al., “From Image Descriptions to Visual Denotations,” *TACL*, 2014.